Annex C

MAKESPEC.M Program Code (*Matlab 4.0*)

```
function makespec(WaveFile,PlotFlag)

% MAKE ME A SPECTROGRAM!!
%
% Usage: makespec('Wave.File',PlotFlag) [PlotFlag=1 displays specgram chunks]
%
% Note- This algorithm builds a spectrogram by segmenting the sound file into manageable
%       lengths and processing each piece sequentially. Each chunk of the spectrogram is
%       concatenated together to form one big spectrogram representing the whole sound file.
%       In this manner, long sound files can be processed with limited RAM.

% SET PLOTS TO WHITE BACKGROUND FOR PRINTING SCREEN SHOTS
close;close;close;close;close;close;close;close;close;close;close;
whitebg;

%LOAD WAVE DATA
[WaveData,Fs]=loadwav(WaveFile);

% CONSTANTS
NFFT=512;                   % Number of samples in FFT window frame
% TimeRes=0.0116;           % Time resolution (in seconds) of each spectrogram timeslice
% NOverLap=(NFFT-round(Fs*TimeRes)); % Number of samples in frame overlap required
to produce desired time resolution
NOverLap=0.5*NFFT;
```

```
FreqRes=Fs/NFFT;            % Bandwidth resolution of FFT frequency bands (specgram
rows)
MaxFreq=4000;               % Ignore frequencies above this level
HighBand=ceil(MaxFreq/FreqRes); % The specgram row representing the maximum
frequency


% MAXIMIZE AMPLITUDE OF WAVE FILE
% 16 bit resolution gives 2^16 amplitude levels, 2^15 positive and 2^15 negative
Boost_Ratio = 2^15/max(abs(WaveData));
WaveData = WaveData * Boost_Ratio;

% DEFINE NUMBER OF SAMPLES TO BE PROCESSED IN EACH CHUNK
% the memory constant in the equation should be adjusted according to how much matlab
%memory is available
%MemoryConstant=100;
%ChunkSize=round(Fs*TimeRes*MemoryConstant);
ChunkSize=Fs; % 1 second worth of samples

% DETERMINE HOW MANY CHUNKS THERE ARE IN THE SOUND FILE
NumberChunks = ceil(length(WaveData)/ChunkSize);


%##########################################################################
%
% BIG LOOP - begin producing spectrograms one chunk at a time
%
%~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

for chunk=1:NumberChunks  %
<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<
```

```
CHUNK=int2str(chunk);

% LOAD NEXT 1 SECOND OF SPEAKER'S VOICE SAMPLE
if chunk < NumberChunks
      ChunkData=WaveData((1+(chunk-1)*ChunkSize):(chunk*ChunkSize));
else
      ChunkData=WaveData((1+(chunk-1)*ChunkSize):length(WaveData));
      % If last chunk then load to End Of File

   % check length of last chunk to prevent a crash - if chunk is too small skip it
   if length(ChunkData) < NFFT
     break;
   end;
end; % of if chunk


% CALCULATE REAL PART OF VOICE CHUNK SPECGRAM
ChunkSpec=abs(specgram(ChunkData,NFFT,NOverLap));

% KEEP ONLY THE BOTTOM HALF OF THE SPECTROGRAM UP TO MAX
FREQUENCY
ChunkSpec=ChunkSpec(1:HighBand,:);

if PlotFlag==1
      % PLOT SPECGRAM CHUNK
      figure(chunk);
      clf reset;
      [SpecRows,SpecCols]=size(ChunkSpec);
      TimeIndex=(1:SpecCols)*NOverLap/Fs;
      FreqIndex=(1:SpecRows)*Fs/NFFT;
      imagesc(TimeIndex,FreqIndex,ChunkSpec); axis('xy')
```

```
        title(['Chunk #',CHUNK,' of ',WaveFile]);
        xlabel('Time (sec)');
        ylabel('Frequency (Hz)');

% CONVERT SCALED COLOUR IMAGE TO INVERTED GRAYSCALE COLOUR %
MAPPING TO ACCOMODATE PRINT OUTS
        colormap(gray);
        graymap=colormap;
        colormap(1-graymap);
        pause(3);

   end;

   % SAVE SPECGRAM CHUNK TO FILE
   % If this is the first chunk, then create a new file otherwise append it to other chunks in
   % existing file
   if chunk==1
        Spec = ChunkSpec;
        save tempfile.mat Spec ;
   else
        load tempfile.mat;
        Spec = [Spec,ChunkSpec];
        save tempfile.mat Spec ;
   end; % if chunk==1

end; % for chunk=1:NumberChunks
<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<

if PlotFlag == 1
        figure(chunk+1)
        clf reset;
```

```
        [SpecRows,SpecCols]=size(Spec);
        TimeIndex=(1:SpecCols)*NOverLap/Fs;
        FreqIndex=(1:SpecRows)*Fs/NFFT;
        imagesc(TimeIndex,FreqIndex,Spec); axis('xy')
        title(['Spectrogram of ',WaveFile]) ;
        xlabel('Time (sec)');
        ylabel('Frequency (Hz)');

% CONVERT SCALED COLOUR IMAGE TO INVERTED GRAYSCALE COLOUR
% MAPPING TO ACCOMODATE PRINT OUTS
        colormap(gray);
        graymap=colormap;
        colormap(1-graymap);
        pause(3);

end;

% SAVE SPECTROGRAM TO "S" FILE
save tempfile Spec NFFT Fs WaveFile Boost_Ratio
clear
load tempfile
save(['s',WaveFile(2:length(WaveFile)-4),'.mat'])
fprintf('\nSpectrogram saved to %s \n',['s',WaveFile(2:length(WaveFile)-4),'.mat'])
```